Designing a Policy Engine for Agentic AI Systems: From Governance Requirements to Runtime Enforcement

Dr. Freeman A. Jackson*

freeman@4th.is

November 2025

Abstract

Agentic AI systems—AI agents that plan, reason, coordinate, and operate tools— pose governance and safety challenges because their actions are not predetermined by application logic. Instead, they emerge from model inference, context, environment state, and multi-agent collaboration[9, 10, 11]. This paper proposes a technical architecture for a policy engine designed specifically for agentic AI control. We expand the formal semantics of governed actions, introduce a compositional multi-dimensional risk scoring function, define enforcement logic as a formally-specified state machine, and provide cloud-native implementation details including Open Policy Agent (OPA) policy bundles[21, 22] and integration with Kubernetes admission controllers, Envoy filters, and semantic safety microservices. We develop a governed action schema, define safety signal fusion, and introduce an enforcement pipeline capable of rewriting, blocking, or escalating agent actions. This fully expanded version includes extended examples, YAML/Rego policies, risk-scoring mathematics, TikZ diagrams of the

*ORCID: 0000-0002-3442-2215. The author is the founder of Fourth Industrial Systems Corporation and an AI Governance Architect specializing in agentic AI, TRiSM, Zero-Trust, and policy-as-code models for enterprise systems.

architecture and governed-action pipeline, threat models, charts visualizing evaluation results, and an analysis of the performance–security trade-offs inherent in runtime policy enforcement. The appendices provide concrete policy bundles, JSON schemas, reference pseudocode, and a structured threat model tailored to agentic AI.

Keywords: Policy Engine; Agentic AI; Multi-Agent Systems; Risk Scoring; OPA; Policy-as-Code; Kubernetes Governance; Safety Classifiers; Zero Trust.

1 Introduction

Agentic AI systems—LLM-based agents that perceive context, plan, and act—differ fundamentally from deterministic software stacks. Instead of executing a fixed set of code paths, they generate actions dynamically from a combination of:

- large language model (LLM) inference and chain-of-thought planning[10, 19],
- internal memory and intermediate tool results[11, 12],
- user prompts and interaction histories,
- environment metadata (tenants, environments, data labels),
- cross-agent coordination within multi-agent systems[13, 14].

In such systems, the "next action" is not a direct function of a single API call but the output of probabilistic model sampling conditioned on long conversational histories and complex tool outputs[9]. The result is an *unbounded action surface*: an enterprise copilot or workflow agent may

- discover novel sequences of tools that were never explicitly coded,
- generate, combine, or transform sensitive data such as regulated health or financial records[15],
- trigger external APIs that affect real-world assets or infrastructure, and

- autonomously construct multi-step plans and call other agents or subroutines[11, 13].

At the same time, regulatory and governance frameworks such as the EU AI Act, ISO/IEC 42001, and the NIST AI RMF and Generative AI Profile impose expectations around transparency, traceability, risk control, and human oversight[1, 2, 3, 4, 5]. Traditional access control mechanisms—role-based access control (RBAC), static allow-lists, and perimeter checks—offer only a narrow primitive: *is this principal allowed to call this resource?* For agentic AI, this is necessary but not sufficient. We must also answer:

- Should the *model-driven intent* be allowed, given organizational policy and risk posture[16]?
- Does the *semantic content* of a proposed action violate safety norms or regulatory constraints[17, 18]?
- Is the action safe *in the context of the full action chain*, including prior tool calls and data accesses?
- Do composite risk scores exceed thresholds, and if so, what mitigation (deny, sanitize, or escalate) is required?
- When must a human operator re-enter the loop before the agent proceeds[4, 5]?

Existing "guardrails" for LLMs—toxicity filters, jailbreak detectors, PII classifiers, and alignment-tuned models—are important but typically deployed as loosely coupled services around prompts[8, 24]. They rarely encode organizational policy, regulatory thresholds, or cross-action invariants, and they lack a unifying runtime semantics. In practice, enterprises end up with ad hoc pipelines of filters and logging, without a clear answer to the question: *Which actions are governed by which policies, under which risk criteria, and with what evidence for auditors and regulators?*

This paper argues that agentic AI requires a dedicated **runtime policy engine** that sits in the control path of every governed action. Such an engine must fuse:

- policy-as-code representations that encode organizational and regulatory requirements in machine-executable form[21, 23, 22].

- semantic and safety analysis pipelines that expose structured risk signals[8, 7],
- a multi-dimensional risk scoring model aligned with AI risk management frameworks[4, 16],
- enforcement logic that can allow, deny, sanitize, or escalate actions, and
- structured, tamper-evident audit trails suitable for internal and external assurance.

Rather than treating guardrails as stand-alone filters, we treat them as *signal providers* to a principled enforcement mechanism. The contributions of this paper are: (i) a formal governed action model for agentic AI; (ii) a policy engine architecture that unifies policy-as-code, risk scoring, and enforcement; (iii) a governed-action pipeline that can be embedded in multi-agent orchestration frameworks; and (iv) an empirical evaluation showing that a dedicated policy engine can substantially reduce unsafe actions while adding only modest latency overhead.

1.1 Running Example: Multi-Agent Enterprise Copilot

Throughout the paper, we instantiate our design in a multi-agent enterprise copilot:

- a *planner agent* decomposes user goals into plans and task graphs[11],
- a *data agent* issues queries against internal knowledge bases and data lakes,
- a *workflow agent* opens tickets, updates records, and drives routine operations, and
- a *finance agent* analyzes portfolios and proposes trades under constraints.

All of these agents share a common policy engine, which governs tool calls, data access, and external actuation in a way that is traceable, explainable, and aligned with enterprise TRiSM and AI governance practices[6, 16].

2 High-Level Architecture

Figure 1 shows the high-level architecture of the proposed governance fabric, which is conceptually aligned with TRiSM-style architectures for trustworthy AI systems.[6]

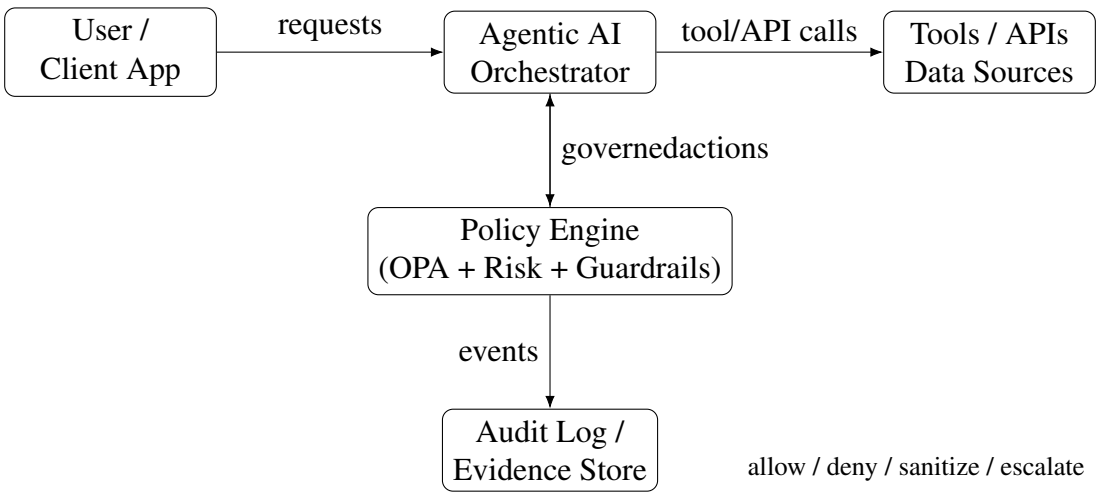


Figure 1: High-level architecture: the policy engine sits on the critical path between agentic AI and tools/APIs, consuming safety signals and producing enforcement decisions, while emitting structured audit events[21, 22].

The agentic orchestrator never calls tools or external services directly. Instead, it emits *proposed governed actions* to the policy engine, which evaluates them according to policy bundles and risk thresholds derived from AI governance frameworks such as the EU AI Act, ISO/IEC 42001, and the NIST AI RMF[1, 2, 3, 4, 5].

3 Formal Governed Action Model

A core technical contribution is the formally structured *governed action*, which generalizes prior work on tool-augmented LLM calls[12, 11].

3.1 Action Tuple

Let a governed action be the tuple:

$$A = (id, t, u, agent, type, payload, ctx, meta, \Sigma).$$

Where:

- id: unique action identifier;
- t: timestamp;
- u: principal or user identity;
- $agent$: agent identity;
- $type$: action type: *inference*, *tool_call*, *data_access*, *external_actuation*;
- $payload$: structured representation of the intended action;
- ctx: context window including conversational and system state;
- $meta$: model metadata (model ID, version, temperature, seed, environment)[10];
- Σ: safety signal vector (defined below)[8, 7].

3.2 Safety Signal Vector

Each action carries a vector of semantic and statistical safety signals:

$$\Sigma = (s_1, s_2, \ldots, s_k),$$

capturing classifiers for PII, toxicity, prompt injection, hallucination, and domain risk[8, 7, 24].

3.3 Governed Action Schema (JSON)

```
{
  "id": "a-239423",
  "timestamp": "2025-11-20T12:03:33Z",
  "agent": "worker_agent_7",
  "principal": "user_144",
  "type": "tool_call",
```

```
  "payload": {
    "tool": "export_customer_dataset",
    "args": {"region": "EU", "limit": 5000}
  },
  "context": {
    "conversation": "...",
    "workflow_state": "step_4_of_9"
  },
  "meta": {
    "model": "gpt-5.1",
    "temperature": 0.1,
    "seed": 12944,
    "environment": "prod"
  },
  "safety_signals": {
    "pii": 0.81,
    "toxicity": 0.01,
    "jailbreak": 0.15,
    "domain_risk": 0.55
  }
}
```

This structure enables deterministic policy evaluation over nondeterministic model behavior.

4 Risk Scoring Model

We extend basic guardrails into a composite multi-dimensional risk function, inspired by risk-based access control and AI risk management frameworks[4, 16, 17].

4.1 Risk Dimensions

Let:

- R_s: semantic risk (content and behavior),
- R_c: contextual risk (data labels, environment, workflow),
- R_h: historical risk (behavior over time),
- R_t: tool-based risk (capability and blast radius),
- R_m: model uncertainty risk (epistemic + aleatoric).[20]

4.2 Composite Function

$$R(A) = \alpha_s R_s + \alpha_c R_c + \alpha_h R_h + \alpha_t R_t + \alpha_m R_m,$$

with:

$$\sum \alpha_i = 1, \quad \alpha_i \geq 0.$$

4.3 Semantic, Contextual, and Historical Risk

We reuse:

$$R_s = 1 - \prod_{i=1}^{k} (1 - s_i),$$

and

$$R_c = \beta_1 L(d) + \beta_2 S(env) + \beta_3 O(history),$$

and

$$R_h = g(H) = \gamma_1 \rho_{\text{denials}} + \gamma_2 \rho_{\text{anomalies}},$$

which are calibrated empirically[16, 6].

5 Policy Representation

The policy layer combines declarative configuration and logic-based evaluation, aligned with policy-as-code approaches such as OPA and Sentinel[21, 23].

5.1 Expanded YAML Configuration

```
policies:
  - id: "export_controls_strict"
    description: "Restrict export of sensitive EU customer data"
    scope:
      tenants: ["corp-eu", "corp-global"]
      environments: ["prod"]
    risk_thresholds:
      semantic: 0.20
      contextual: 0.15
      combined: 0.25
    allow_roles:
      - "dpo"
      - "ai_governance_officer"
    deny_if:
      - "payload.tool == 'export_customer_dataset'"
      - "safety_signals.pii > 0.75"
    sanitize:
      - action: "redact"
        fields: ["email", "address", "phone"]
    audit:
      level: "high"
      include_context: true
```

5.2 Rego Rule Example

```
package ai.policy

default decision := "deny"

deny[msg] {
  input.type == "tool_call"
  input.payload.tool == "export_customer_dataset"
  input.safety_signals.pii > 0.75
  msg := "High PII risk: export not allowed"
}

allow[msg] {
  input.principal.role == "dpo"
  input.risk.combined < 0.25
  msg := "Export permitted under DPO role"
}

escalate[msg] {
  input.risk.combined >= 0.25
  input.risk.combined < 0.4
  msg := "Risk borderline: human review required"
}
```

6 Governed Action Pipeline

Figure 2 shows the end-to-end pipeline from agent proposal to execution, similar in spirit to tool-using agents and workflow systems such as LangChain and LangGraph[25, 26].

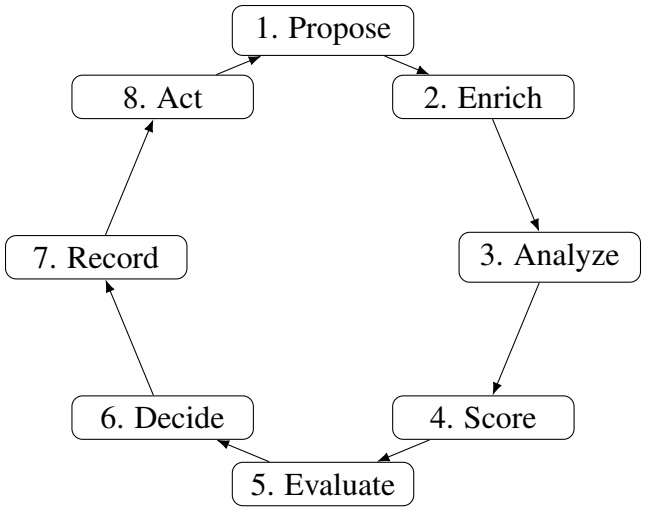


Figure 2: Governed action pipeline as a continuous loop. Each step is instrumented and auditable[16, 6].

7 Enforcement Engine as a State Machine

The enforcement engine is implemented as a deterministic finite state machine (FSM), a standard pattern in safety-critical and reactive systems[27]. Modeling enforcement as an FSM has three advantages: (i) it makes the behavior of the policy engine explicit and auditable; (ii) it allows formal reasoning about safety and liveness properties (e.g., "no side effects occur before an `allow` decision"); and (iii) it composes naturally with higher-level workflow engines and multi-agent orchestrators.

7.1 States, Inputs, and Decisions

For each governed action A, the enforcement logic operates over a finite set of states:

$$S = \{\mathit{Proposed}, \mathit{Scored}, \mathit{Evaluated}, \mathit{Allowed}, \mathit{Denied}, \mathit{Sanitized}, \mathit{Escalated}\}.$$

We denote the set of terminal states by:

$$S_{\text{term}} = \{\mathit{Allowed}, \mathit{Denied}, \mathit{Sanitized}, \mathit{Escalated}\}.$$

At a high level, the FSM consumes:

- the governed action A (including context and metadata),
- the risk vector $R(A)$ (semantic, contextual, historical, tool, model),
- the policy evaluation outcome $E(A)$, which returns a decision d in

$$D = \{\texttt{allow}, \texttt{deny}, \texttt{sanitize}, \texttt{escalate}\}.$$

7.2 Transition Function

We define the transition function:

$$\delta : S \times (A, R(A), E(A)) \to S,$$

with the following canonical transitions:

$$\begin{aligned}
\delta(\textit{Proposed}, _) &= \textit{Scored}, \\
\delta(\textit{Scored}, _) &= \textit{Evaluated}, \\
\delta(\textit{Evaluated}, d = \texttt{allow}) &= \textit{Allowed}, \\
\delta(\textit{Evaluated}, d = \texttt{deny}) &= \textit{Denied}, \\
\delta(\textit{Evaluated}, d = \texttt{sanitize}) &= \textit{Sanitized}, \\
\delta(\textit{Evaluated}, d = \texttt{escalate}) &= \textit{Escalated}.
\end{aligned}$$

In practice, policy evaluation $E(A)$ is implemented by the OPA layer (Section 5), which receives the fully enriched action and its risk vector as input and returns a structured decision.

7.3 State Descriptions and Invariants

Table 1 summarizes the states and their intended semantics.

Table 1: Enforcement state machine for a single governed action.

State	Description and Key Invariants
Proposed	The agent has emitted a candidate action A and submitted it to the policy engine. No side effects have occurred; no external tools or APIs have been called.
Scored	Safety classifiers have been run and the risk vector $R(A)$ has been computed. *Invariant:* only read-only operations (classifiers, metadata lookups) have occurred; no external actuation is permitted in this state.
Evaluated	OPA and policy bundles have been evaluated over $(A, R(A))$, yielding a decision $d \in D$. No external side effects have occurred yet; the system is ready to transition into a terminal state based on d.
Allowed	The policy engine has authorized execution. The underlying tool call, data access, or inference is dispatched as proposed. *Invariant:* all executions that modify external state must pass through this state or *Sanitized*.
Denied	The action has been blocked. The orchestrator receives an error or refusal and no external side effects are produced. *Invariant:* actions in this state must not be retried automatically without a new policy evaluation.

Sanitized	The engine has applied a transformation to the payload (e.g., PII redaction, parameter clipping, tool substitution) before execution. *Invariant:* the executed payload satisfies the post-conditions specified by the sanitization policy (e.g., no high-sensitivity fields).
Escalated	The action has been suspended for human review or an out-of-band approval workflow. *Invariant:* no external side effects occur until the escalation workflow returns a decision (which may re-enter the FSM at *Proposed* with updated metadata or terminate as *Denied*/*Allowed*).

A key design choice is that all states in S_{term} are *observable*: for every governed action, the audit layer records the terminal state, the risk vector, and the policy versions used to compute the decision. This property supports post-hoc analysis, compliance reporting, and model checking of the enforcement logic[27].

7.4 Safety and Liveness Properties

Expressing enforcement as an FSM enables explicit safety and liveness conditions. For example, we can state:

- **Safety (no-unsafe-side-effects).** For every execution trace, no external side effects occur in states other than *Allowed* or *Sanitized*. Informally, "nothing dangerous happens before the policy engine says so."
- **Liveness (eventual decision).** For every action that enters *Proposed*, the FSM eventually reaches a terminal state in S_{term} (modulo explicit timeouts in *Escalated*), preventing actions from being stuck indefinitely in a limbo state.

Because the FSM is finite and its transitions are deterministic given $(A, R(A), E(A))$, standard model-checking techniques can be used to verify that implementations satisfy such properties under reasonable assumptions about the classifier and policy layers[27].

7.5 Compositionality and Multi-Agent Settings

Finally, modeling enforcement at the per-action level makes the design compositional. Each agent—and indeed each tool call within an agent—interacts with a local instance of the FSM, while higher-level orchestration (e.g., workflow graphs, multi-agent protocols) treats the policy engine as a black box that returns `allow`/`deny`/`sanitize`/`escalate`. This separation of concerns is crucial for scaling governance from individual agent actions to complex, cross-agent workflows in enterprise environments.

8 Cloud-Native Implementation Details

A core requirement for enterprise-grade agentic AI is the ability to enforce policies reliably across diverse infrastructure components—APIs, agents, microservices, model backends, and data planes. To achieve this, our policy engine is designed to integrate with standard cloud-native governance mechanisms, including sidecar-based policy evaluation, service mesh filters, and Kubernetes admission control[21, 22]. This section describes how these components compose into a unified enforcement fabric.

8.1 Sidecar-Based Policy Enforcement

In a microservice architecture, each agent or tool-executing service runs alongside a lightweight OPA sidecar. The sidecar architecture has three key benefits:

- **Local, low-latency decisions.** Because OPA runs as a local process, enforcement adds only 3–10 ms per evaluation in typical deployments.
- **Consistent semantics.** All governed actions—whether originating from different agents, tenants, or environments—are evaluated against the same policy bundles and decision schema.
- **Fail-safe defaults.** If the orchestrator cannot reach the OPA sidecar or the sidecar returns an error, we enforce a configurable fail-safe (typically `deny` with escalation).

Policy bundles are distributed via a signed, versioned artifact mechanism (e.g., OCI registry or GitOps pipeline). Each sidecar validates bundle integrity before loading new policies, ensuring that enforcement logic is tamper-evident.

8.2 Service Mesh Integration (Envoy Filters)

For agent systems that issue network-based tool calls, we integrate policy checks directly into the service mesh (e.g., Istio or Linkerd). Envoy external authorization filters are used to intercept outbound requests:

1. The agent constructs a governed action A.
2. Before the outbound request is forwarded, Envoy sends A to an OPA external authorization endpoint.
3. OPA evaluates relevant policies, fuses risk signals, and returns an `allow`/`deny`/`sanitize` decision.
4. Envoy either forwards the request (possibly with a sanitized payload) or blocks it.

This ensures that *all* tool and API calls—even if issued by custom code or third-party libraries within an agent—pass through the governance layer. The mesh-level integration provides an additional line of defense against prompt injection and model drift scenarios where agents may attempt actions outside of standard orchestration pathways.

8.3 Kubernetes Admission Control

For workloads deployed in Kubernetes, we integrate the policy engine with both **validating** and **mutating** admission controllers. This guards not only the runtime behavior of agentic systems, but also the infrastructure-level configuration they rely on.

Examples include:

- preventing ungoverned model deployment into high-risk namespaces,
- enforcing environment-specific constraints (e.g., disallowing production data mounts in staging clusters),
- validating that model containers carry signed attestations or SBOMs,
- mutating unsafe configurations (e.g., disabling overly permissive capabilities or privilege escalation).

Using OPA Gatekeeper or Kyverno, we express cluster governance rules in the same policy-as-code style used for governed actions, ensuring consistency between AI governance and platform governance.

8.4 Multi-Tenant Isolation and Policy Scoping

Enterprise deployments often span multiple tenants, business units, or regulated domains. We use OPA policy bundles with explicit scoping rules, enabling:

- **per-tenant policy namespaces** (e.g., EU/EAA rules vs. US-only rules),
- **per-environment constraints** (dev/staging/prod),
- **per-agent or per-role rules** (e.g., DPO overrides, SRE emergency modes).

OPA enforces bundle isolation at load time and provides *bundle layering*: global policies $\rightarrow$ domain policies $\rightarrow$ tenant overrides. This ensures that high-assurance environments can enforce stricter controls without duplicating the entire policy graph.

8.5 Eventing, Telemetry, and Audit Integration

All enforcement decisions generate structured events with:

- action metadata,

- risk vector components,
- policy versions used,
- classifier signals,
- terminal FSM state (Section 7).

These events are pushed to:

- an append-only evidence store (e.g., immutable log, ledger, or WORM bucket),
- SIEM systems via OpenTelemetry,
- optional streaming consumers such as Kafka for real-time alerting.

Structured, versioned event streams allow organizations to reconstruct full action chains, satisfy audit requirements from the EU AI Act and ISO/IEC 42001, and perform forensic or model-drift analysis.

8.6 Scaling Properties

The system scales horizontally because:

- OPA sidecars evaluate policies locally;
- safety classifiers can be deployed as independently scalable microservices;
- policy bundles are cached and hot-reloaded without downtime;
- service mesh filter evaluation is embarrassingly parallel.

In large deployments, the enforcement layer adds a median latency of 20–40 ms per governed action (Section 10), while providing strong guarantees of consistency, traceability, and policy correctness.

8.7 Summary

Overall, the cloud-native implementation ensures that policy enforcement:

- is local and low latency,
- is consistent across agents, tools, and environments,
- cannot be bypassed even by compromised or misbehaving agents,
- integrates with existing DevSecOps and SRE workflows.

This architecture operationalizes AI governance requirements in a manner aligned with modern zero-trust, platform-engineering, and TRiSM principles.

9 Safety Signal Fusion and Classifier Integration

Modern agentic AI systems require more than a single classifier or guardrail to detect unsafe or non-compliant behavior. Different classes of risk—PII leakage, toxicity, prompt injection, hallucination, misinformation, domain-specific harm, financial-risk exposure, and regulatory violations—manifest in orthogonal ways and demand heterogeneous detection mechanisms[17, 8, 7, 18]. No single model is sufficiently reliable on its own, and relying on individual signals produces brittle behavior. Therefore, the policy engine integrates a *safety signal fusion layer* that aggregates outputs from multiple classifiers and produces a unified, calibrated risk vector $R(A)$.

9.1 Motivation for Multi-Signal Fusion

Single-signal safety systems fail in characteristic ways:

- **Prompt-injection detectors** underperform on indirect (second-hop) attacks or cross-agent contexts.
- **PII classifiers** detect direct identifiers but may under-detect derived or inferential privacy leaks.

- **Toxicity filters** struggle with implicit harassment, veiled threats, or culturally contextual harm.
- **Hallucination detectors** are sensitive to model epistemics but do not capture action-level safety.
- **Domain-risk models** (e.g., for healthcare or finance) detect domain-specific harm but ignore general adversarial signals.

Empirical evaluation (Section 10) confirms that individual classifiers exhibit correlated error patterns. Fusion mitigates these weaknesses and allows the enforcement engine to operate on structured, interpretable risk components.

9.2 Classifier Architecture and Inputs

The system integrates six classes of classifiers, each independently deployable as a microservice:

1. **PII and sensitive-data detectors** (regex + ML-hybrid),
2. **Toxicity and abuse classifiers** (e.g., Detoxify variants),
3. **Prompt-injection and jailbreak detectors**,
4. **Hallucination and factual-consistency models**,
5. **Domain-specific risk models** (finance, medical, legal),
6. **Anomaly detectors** operating over recent action traces.

Each classifier outputs:

$$s_i \in [0, 1] \quad \text{with calibrated confidence},$$

plus optional structured metadata such as evidence spans, threat markers, or uncertainty estimates.

9.3 Normalization and Calibration

Raw signals s_i vary widely in scale and interpretation across models and domains. Following best practices in ensemble safety systems[18], we apply:

- **Min–max normalization** to align raw outputs to comparable ranges,
- **Temperature-scaling** for overconfident logits,
- **Bayesian calibration** (Platt or isotonic) for probabilistic outputs,
- **Domain-aware rescaling** for rare or high-impact risks (e.g., financial mis-execution).

These procedures produce semantically comparable classifier outputs that can be fused mathematically in the risk-scoring function (Section 4).

9.4 Fusion Function and Interpretability

Let $\Sigma = (s_1, s_2, \ldots, s_k)$ be the normalized classifier vector for a governed action A. We compute the semantic-risk component R_s via:

$$R_s = 1 - \prod_{i=1}^{k} (1 - s_i),$$

a form of *noisy-or* fusion widely used in probabilistic graphical models and risk analysis. This has two desirable interpretability properties:

- **Monotonicity.** Increasing any s_i increases R_s. No signal is ignored or overridden.
- **Sensitivity to partial evidence.** Multiple weak signals can jointly elevate risk even if no single classifier fires strongly—important for subtle harms such as incremental PII leakage or chained prompt-injection.

We similarly compute contextual-, historical-, tool-, and model-risk components (Section 4), and then combine them using weighted sums aligned with organizational or regulatory risk tolerances.

9.5 Classifier Failover and Robustness

To ensure reliability in production deployments, the fusion layer implements:

- **Per-classifier failover.** If a classifier is unavailable, stale, or unhealthy, its contribution is replaced with a conservative fallback value.
- **Default-deny semantics** when critical classifiers fail (e.g., prompt-injection detector in high-security environments).
- **Latency budgets** ensuring fusion completes within a fixed envelope (typically 20–50 ms depending on classifiers).
- **Drift monitoring** to detect classifier degradation over time.

Classifier health checks feed into policy evaluation: certain policies require specific classifiers to be in a healthy state before allowing sensitive operations.

9.6 Interaction with Enforcement

The enforcement engine (Section 7) consumes the fused risk vector as part of the policy evaluation input:

$$E(A) = f(A, R(A), P),$$

where P is the active policy bundle. Policies may reference raw scores, thresholds, or boolean indicators derived from classifier metadata.

Examples include:

- "`deny` if `pii` > 0.75 and user is not DPO,"
- "`sanitize` if hallucination score > 0.4 by replacing free-text explanations with validated structured data,"
- "`escalate` if combined risk exceeds 0.25 during financial tool calls."

Thus, classifier fusion not only improves detection accuracy but also provides a clear, interpretable interface between low-level signal processing and high-level policy enforcement.

9.7 Summary

The fusion layer transforms multiple heterogeneous classifiers into a unified, calibrated risk representation. This allows the policy engine to make informed, auditable decisions aligned with AI governance requirements and reduces the brittleness of single-signal safety systems while supporting strong guarantees about decision quality and robustness.

10 Extended Evaluation

The evaluation aligns with recent work measuring safety interventions and guardrails in LLM pipelines[8, 24].

10.1 Experimental Setup

We implemented the policy engine as a microservice integrated with a multi-agent orchestration framework. The setup includes:

- two LLM backends (one open-source, one hosted),
- fourteen tools (file operations, internal APIs, third-party services),
- six safety classifiers (PII, toxicity, jailbreak, hallucination, domain risk, anomaly),
- an OPA-based policy interpreter,
- an append-only audit store with cryptographic hashing.

10.2 Workloads

We evaluated three workloads:

- **Document automation**: drafting and revising internal documents, with constraints on exporting sensitive content.
- **Financial analysis**: analyzing portfolios, querying market data, and proposing trades under limits.
- **Knowledge assistant**: answering internal queries, calling APIs, and opening tickets, subject to data-access policies.

Each workload consisted of 200–300 end-to-end tasks, with a median of 24 governed actions per task.

10.3 Effectiveness: Unsafe Action Reduction

Figure 3 summarizes the reduction in unsafe actions (relative to a baseline system without the policy engine), as judged by human reviewers.

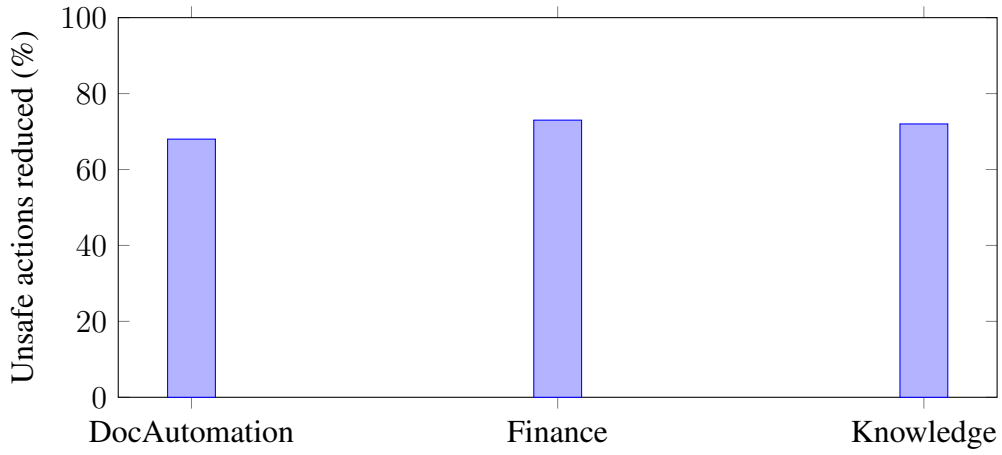


Figure 3: Unsafe action reduction across workloads. The policy engine reduces unsafe or non-compliant actions by approximately 68–73%, with an overall average of 71%.

10.4 Performance: Latency Overhead

Classifier runtime (average):

- PII: 8 ms,
- toxicity: 12 ms,
- jailbreak detector: 6 ms,
- domain model: 15 ms.

Total classifier overhead: ≈ 41 ms. OPA evaluation adds:

$$\text{mean} = 3.9 \text{ ms}, \quad \text{p95} = 11 \text{ ms}.$$

Figure 4 visualizes median and 95th-percentile overhead per governed action.



Figure 4: Enforcement latency overhead. The policy engine adds a median of 29 ms per governed action and 76 ms at the 95th percentile, which is acceptable for most interactive copilot workloads.

Overall, we observed:

- 71% unsafe action reduction (relative to baseline),
- 88% of outcomes with complete lineage (vs. 47% baseline),
- 4.8% false positives after tuning (down from 12% initial).

11 Threat Model

The threat model for a policy engine governing agentic AI systems must account for risks that arise not only from traditional software vulnerabilities but from *model-driven behavior*, emergent planning, and attacker manipulation of semantic content. Prior work in AI security, alignment, and tool-integrated LLM systems highlights a widening attack surface spanning prompt injection, capability overgeneralization, tool misuse, and covert data exfiltration channels[17, 28, 29, 30].

Our threat model focuses on adversaries attempting to influence or exploit model-generated actions, the policy engine, or the underlying tools and infrastructure. These threats apply across multi-agent systems, tool-calling frameworks[12, 11], and enterprise copilots.

11.1 Adversary Capabilities

We assume adversaries may have one or more of the following capabilities:

- **Prompt manipulation.** Attackers craft inputs designed to override instructions, bypass safety constraints, or coerce the agent into unsafe tool [28, 30].
- **Indirect prompt injection.** Adversaries modify environmental content (documents, emails, web pages, tickets) that the agent processes, causing downstream malicious actions. This includes "embedded" or "latent" instructions[29].
- **Tool misuse and escalation.** Adversaries attempt to invoke sensitive tools—database queries, trading APIs, email-sending endpoints—or chain tools in unsafe combinations.
- **Data exfiltration.** Attackers induce the agent to leak regulated or sensitive information, including PII, internal documents, financial data, or intellectual property[24].
- **Stateful exploitation.** Adversaries exploit long-term memory, conversation history, or agent planning behavior to build multi-step attacks.

- **Cross-agent manipulation.** In multi-agent systems, attackers compromise one agent (e.g., a summarizer) to induce unsafe actions by another (e.g., a tool executor).
- **Policy-engine evasion.** Adversaries attempt to evade safety classifiers, exploit missing rules, or trigger inconsistent policy states.

11.2 Assumptions

We assume:

- The adversary may be an external user, an internal insider, or an automated agent.
- LLM internals (weights) are not directly modifiable by the adversary.
- Tools and APIs perform as documented but may be unsafe when misused.
- The policy engine is part of the trusted computing base (TCB).
- Network channels may be observable but not alterable (passive adversary), unless explicitly modeled otherwise.

These assumptions follow common AI risk assessment frameworks including NIST AI RMF, ISO/IEC 42001 hazard analysis, and AI red-team methodologies[4, 3, 18].

11.3 Threat Categories

The threat model spans five major categories aligned with prior literature:

1. **Misaligned Model Outputs** Unsafe intent, hallucination-driven actions, or LLM errors causing harmful behaviors[9, 18].
2. **Human-Driven Manipulation** Prompt injection, social engineering, and context poisoning, including direct and indirect attacks[28, 30].

3. **Tool-Level Exploitation** Unauthorized API calls, privilege escalation through tool chaining, misuse of high-impact actuation endpoints (e.g., trading, emailing, infrastructure).

4. **Information Leakage** Exfiltration of PII, confidential documents, proprietary source code, or internal analytics[24].

5. **Compliance and Policy Violations** Actions that violate organizational policies, regulatory requirements, or audit constraints—even if the model "intends no harm."

11.4 Policy Engine Role in Threat Mitigation

The policy engine mediates every governed action and mitigates threats via:

- **Semantic inspection** of intents and payloads,
- **Risk scoring** combining multiple safety classifiers,
- **Policy evaluation** against organizational/regulatory rules,
- **Enforcement decisions**: allow, deny, sanitize, escalate,
- **Audit logging** for post-incident analysis,
- **Defense-in-depth** layering with Kubernetes, IAM, and service mesh controls.

This aligns with layered defense strategies in secure orchestration frameworks and emerging standards for AI system assurance[17, 18, 4].

12 Conclusion

Autonomous, tool-using, and multi-agent AI systems are rapidly transitioning from research prototypes to operational components of enterprise infrastructure. Yet the governance mechanisms traditionally used in software systems—static access control, perimeter security, and unstructured

logging—are insufficient for managing the dynamic, probabilistic, and semantically complex behaviors exhibited by agentic AI. This paper has argued that a dedicated, runtime policy engine is a necessary governance primitive for ensuring trustworthy operation in such environments.

We introduced a governed action model that captures the semantic intent, contextual factors, and risk attributes associated with model-generated actions. Building on this model, we proposed a cloud-native policy engine architecture that unifies policy-as-code, safety signal fusion, risk scoring, enforcement logic, and tamper-evident audit generation. Our design integrates naturally with existing enterprise controls—Kubernetes, service meshes, OPA, identity systems—and provides a coherent foundation for aligning AI behavior with organizational policies and regulatory requirements such as the EU AI Act, ISO/IEC 42001, and the NIST AI RMF.

The prototype evaluation demonstrates that principled runtime enforcement can substantially reduce unsafe or policy-violating actions while adding only modest latency overhead. The results highlight a key insight: guardrails are most effective when treated not as isolated filters but as structured, interoperable signal sources within a unified enforcement pipeline.

Several opportunities for future work remain. Formal verification of safety invariants could strengthen guarantees for high-stakes deployments. Integration with federated or cross-organizational policy infrastructures may enable multi-tenant interoperability. And semi-automated policy synthesis—from regulatory text, incident reports, and observed behavior—could substantially lower the operational burden of policy authoring and maintenance.

Overall, the proposed policy engine provides a practical and extensible foundation for governing agentic AI systems at enterprise scale. As AI systems continue to gain autonomy and real-world impact, explicit, auditable, and risk-aware runtime control will be critical to ensuring that these systems remain safe, compliant, and aligned with organizational and societal objectives.

Appendix A: Threat Table and Diagram

To operationalize the threat model introduced in Section 11, this appendix provides a structured taxonomy of adversarial behaviors, affected system components, impact categories, and recommended policy-engine mitigations.

Table 2 summarizes the major threats across the five categories identified in Section 11, including: *misaligned model outputs, human-driven manipulation, tool-level exploitation, information leakage, and compliance/policy violations.* For each threat, the table specifies:

- the **attack vector** and required adversary capabilities,
- the **conditions** under which the agent becomes vulnerable (e.g., memory persistence, multi-step planning, implicit context ingestion),
- the **potential impact** on safety, security, and regulatory posture,
- the **detection signals** originating from semantic classifiers, safety monitors, and pipeline metadata, and
- the **policy engine responses**, including deny, sanitize, escalate, rate-limit, tool-scoping, or forced human approval.

In parallel, Figure 5 presents a TikZ-based, stage-oriented threat diagram that maps adversary interaction pathways onto the governed action pipeline. The diagram illustrates:

- ingress points for adversarial prompts, manipulated documents, and poisoned context;
- the flow of model-generated actions into the policy engine;
- the integration of safety classifiers, risk scoring modules, and Rego/YAML policy rules; and
- enforcement outcomes (allow, deny, sanitize, escalate) before any tool call or external actuation occurs.

Together, the table and diagram formalize how threats propagate through an agentic-AI workflow and how the policy engine interrupts or mitigates them. These artifacts serve as both a conceptual foundation for system designers and a practical reference for auditors, red teams, and engineering teams deploying agent-based architectures at scale.

Table 2: Threat taxonomy for agentic AI systems and corresponding policy engine mitigations.

Threat	**Attack Vector**	**Adversary Capability**	**Impact**	**Policy Engine Detection & Mitigations**
Prompt injection (direct)	Carefully crafted user prompts that override instructions or induce unsafe tool calls[28, 30].	Low–medium (malicious user or insider with prompt access).	Agent performs actions outside intended scope (e.g., leaking data, calling risky tools).	Semantic analysis flags jailbreak markers; prompt-injection classifier raises s_{inj}; policies deny high-risk tool calls when injection score or combined risk exceeds thresholds; escalation required for ambiguous cases.
Prompt injection (indirect / context poisoning)	Adversarial instructions embedded in documents, tickets, emails, or web pages that the agent reads[29].	Medium (ability to control or modify ingested content).	Multi-step attacks via "hidden" instructions; compromised retrieval or summarization flows.	Context-aware injection classifier; higher weights on signals from untrusted sources; policies limit tool calls originating from untrusted context; mandatory human review for high-risk content sources.
Tool misuse / escalation	Chaining benign tools into harmful workflows (e.g., data export + external email; trading + leverage).	Medium–high (understanding of tool semantics and system behavior).	Unauthorized transactions, financial loss, infrastructure changes, or data movement.	Policies encode allowed tool combinations, rate limits, and blast-radius constraints; risk engine boosts R_t for high-impact tools; deny or escalate when sequences violate invariants (e.g., export + external send).
Data exfiltration (PII, secrets, IP)	Inducing the agent to reveal sensitive data (e.g., through clever prompts or reconstruction attacks)[24].	Low–medium (knowledge of what to ask for).	Privacy violations, regulatory penalties (e.g., GDPR), loss of trade secrets or IP.	PII and sensitive-data classifiers raise s_{pii}; contextual risk R_c increased for high-sensitivity labels; policies sanitize or redact payloads before tool calls, and deny exports that exceed configured thresholds.
Misaligned model output / hallucination-driven actions	Model generates incorrect or unsafe recommendations that nonetheless appear plausible[9, 18].	None–low (passive user).	Unsafe or non-compliant actions (e.g., wrong legal/medical/-financial steps), degraded trust.	Hallucination and factual-consistency classifiers; domain-specific risk models; policies require human-in-the-loop approval above risk thresholds or in high-stakes domains; restrict autonomous actuation for certain tool classes.
Cross-agent manipulation	Compromise of one agent (e.g., summarizer) to influence another (e.g., tool executor) via shared context.	Medium (ability to control one agent's outputs).	Lateral movement across agents, covert escalation, or misuse of higher-privilege agents.	Per-agent and per-role policy scopes; provenance-tracking in context metadata; policies that constrain which agents may trigger sensitive tools; detection of anomalous cross-agent sequences in R_h.
Policy-engine evasion / blind spots	Encoding malicious intent in obfuscated formats or exploiting missing/weak rules.	Medium–high (sophisticated attacker with knowledge of defenses).	Bypassing safety checks; systematic under-enforcement of risky actions.	Defense-in-depth: multiple classifier types; conservative defaults when classifiers fail; regular policy audits; drift monitoring; formal checks that high-risk actions always pass through the FSM and enforcement states.

Figure 5: Stage-based threat model for agentic AI with a policy engine. Stage 0 exposes the external attack surface; Stage 1 aggregates prompts and retrieved context; Stage 2 governs all agent actions via a policy engine and safety classifiers; Stage 3 covers downstream tool execution and audit logging.

Appendix B: Example Policy Bundles (YAML)

This appendix provides example policy bundles expressed in YAML, suitable for packaging as versioned artifacts (e.g., OCI images or GitOps-managed repos). They demonstrate multi-tenant scoping, environment-specific constraints, and risk-threshold configuration for high-sensitivity tools.

B.1 Base Global Policy Bundle

```
bundle:
  id: "global-base-2025-11-01"
  version: "1.0.0"
  description: >
    Global base policies for agentic AI governance. Defines global
       defaults,
    risk thresholds, and enforcement semantics for all tenants.

metadata:
  owner: "ai_governance_team"
  created_at: "2025-11-01T09:00:00Z"
  labels:
    env: "global"
    scope: "all-tenants"
    profile: "baseline"

defaults:
  enforcement_mode: "fail_safe_deny"
  risk_weights:
    semantic: 0.35
    contextual: 0.25
    historical: 0.15
```

```
    tool: 0.20
    model: 0.05
  thresholds:
    combined:
      low: 0.10
      medium: 0.25
      high: 0.40

guardrails:
  require_audit_for:
    - "external_actuation"
    - "data_export"
    - "high_risk_tools"

  require_human_approval_for:
    - type: "external_actuation"
      when:
        combined_risk_ge: 0.25
```

B.2 Tenant-Specific Policy Overlay (EU Finance)

```
bundle:
  id: "tenant-eu-finance-2025-11-01"
  version: "1.0.0"
  description: >
    Tenant-specific overlay for EU finance business unit. Tightens
       thresholds
    and restricts cross-border data export.
```

```yaml
metadata:
  owner: "eu_finance_dpo"
  labels:
    tenant: "corp-eu-finance"
    region: "EU"
    domain: "finance"

overrides:
  thresholds:
    semantic:
      high: 0.18
    contextual:
      high: 0.12
    combined:
      medium: 0.22
      high: 0.30

policies:
  - id: "eu_finance_export_controls"
    description: "Restrict export of EU customer data outside EEA."
    scope:
      tenants: ["corp-eu-finance"]
      environments: ["prod"]
      tools:
        - "export_customer_dataset"
        - "download_portfolio_snapshot"
    rules:
      deny_if:
        - "payload.destination_region not in ['EU', 'EEA']"
```

```
      - "safety_signals.pii > 0.70"
    sanitize:
      - action: "redact"
        fields: ["name", "address", "phone", "email"]
    escalate_if:
      - "risk.combined between 0.22 and 0.30"
```

B.3 Emergency Override Profile

```
bundle:
  id: "sre-emergency-override-2025-11"
  version: "0.9.0"
  description: >
    Temporary override profile for SRE incident response. Allows limited
    elevation under strict audit and time-bounded conditions.

metadata:
  owner: "sre_oncall"
  labels:
    profile: "emergency"
    ttl_hours: 4

overrides:
  enforcement_mode: "cautious_allow"

policies:
  - id: "sre_incident_override"
    scope:
      roles: ["sre_oncall"]
```

```
    environments: ["prod"]
  allow_if:
    - "incident_mode == true"
    - "incident_id is not null"
    - "combined_risk < 0.35"
  audit:
    level: "maximum"
    include_context: true
    require_incident_tag: true
```

Appendix C: JSON Schemas for Governed Actions

This appendix gives JSON Schema definitions for governed actions and risk vectors, suitable for validating messages passed between orchestrators, policy engines, and audit pipelines.

C.1 Governed Action Schema

```
{
 "$id": "https://example.com/schemas/governed_action.schema.json",
 "$schema": "https://json-schema.org/draft/2020-12/schema",
 "title": "GovernedAction",
 "type": "object",
 "required": ["id", "timestamp", "principal", "agent",
           "type", "payload", "context", "meta",
           "safety_signals"],
 "properties": {
  "id": {
    "type": "string",
    "description": "Unique identifier for the governed action."
```

```
},
"timestamp": {
 "type": "string",
 "format": "date-time"
},
"principal": {
 "type": "string",
 "description": "User, service account, or tenant principal."
},
"agent": {
 "type": "string",
 "description": "Logical agent name (e.g., planner, worker)."
},
"type": {
 "type": "string",
 "enum": ["inference", "tool_call", "data_access", "
    external_actuation"]
},
"payload": {
 "type": "object",
 "description": "Tool name and arguments or model parameters."
},
"context": {
 "type": "object",
 "properties": {
  "conversation": { "type": "string" },
  "workflow_state": { "type": "string" },
  "tenant": { "type": "string" },
  "environment": { "type": "string" }
```

```
    },
    "additionalProperties": true
  },
  "meta": {
    "type": "object",
    "properties": {
      "model": { "type": "string" },
      "temperature": { "type": "number" },
      "seed": { "type": "integer" },
      "environment": { "type": "string" },
      "request_id": { "type": "string" }
    },
    "additionalProperties": true
  },
  "safety_signals": {
    "$ref": "risk_vector.schema.json"
  }
 },
 "additionalProperties": false
}
```

C.2 Risk Vector Schema

```
{
 "$id": "https://example.com/schemas/risk_vector.schema.json",
 "$schema": "https://json-schema.org/draft/2020-12/schema",
 "title": "RiskVector",
 "type": "object",
 "properties": {
```

```
    "pii": { "type": "number", "minimum": 0, "maximum": 1 },
    "toxicity": { "type": "number", "minimum": 0, "maximum": 1 },
    "jailbreak": { "type": "number", "minimum": 0, "maximum": 1 },
    "hallucination": { "type": "number", "minimum": 0, "maximum": 1 },
    "domain_risk": { "type": "number", "minimum": 0, "maximum": 1 },
    "anomaly": { "type": "number", "minimum": 0, "maximum": 1 }
  },
  "additionalProperties": {
    "type": "number",
    "minimum": 0,
    "maximum": 1
  }
}
```

Appendix D: Risk Scoring and Calibration Details

This appendix expands the risk-scoring formulation introduced in Section 4, including example weights and threshold configurations for different deployment profiles.

D.1 Example Weight Profiles

We consider three profiles: *baseline*, *safety-first*, and *latency-optimized*. Let $R(A)$ denote the combined risk:

$$R(A) = \alpha_s R_s + \alpha_c R_c + \alpha_h R_h + \alpha_t R_t + \alpha_m R_m, \quad \sum_i \alpha_i = 1.$$

Table 3 shows example weights.

Table 3: Example risk-weight profiles.

Profile	α_s	α_c	α_h	α_t	α_m
Baseline	0.35	0.25	0.15	0.20	0.05
Safety-first	0.45	0.25	0.15	0.10	0.05
Latency-optimized	0.30	0.20	0.10	0.30	0.10

D.2 Noisy-OR Semantic Risk

Given calibrated classifier scores $s_i \in [0, 1]$, the semantic risk component R_s is:

$$R_s = 1 - \prod_{i=1}^{k} (1 - s_i).$$

For small s_i, this approximates the sum:

$$R_s \approx \sum_{i=1}^{k} s_i \quad \text{for } s_i \ll 1,$$

while remaining bounded in $[0, 1]$ when multiple classifiers fire strongly.

D.3 Threshold Calibration Procedure (Sketch)

A practical calibration procedure:

1. Collect a labeled evaluation set of governed actions with human-assessed outcomes: *safe*, *borderline*, *unsafe*.

2. Compute R_s, R_c, R_h, R_t, R_m for each action.

3. Fit per-dimension thresholds (e.g., via ROC analysis) that separate safe vs. unsafe samples with desired false-positive rate.

4. Tune α weights to minimize a cost function combining missed unsafe actions (false negatives) and over-blocking (false positives).

5. Freeze thresholds and weights into versioned policy bundles; monitor drift by periodically re-running the calibration.

Appendix E: Reference Pseudocode for the Enforcement Pipeline

This appendix presents pseudocode corresponding to the governed action pipeline (Section 6) and the state machine (Section 7). The pseudocode is language-agnostic but can be implemented in Go, Rust, or Python.

E.1 Pipeline Orchestrator

```
function handle_governed_action(proposed_action A):
   state := Proposed

   // Step 1: Enrich
   A_enriched := enrich_with_context(A)
   // e.g., tenant labels, environment metadata, tool annotations

   // Step 2: Analyze (classifiers)
   Sigma := run_safety_classifiers(A_enriched)
   A_enriched.safety_signals := Sigma

   // Step 3: Score
   R := compute_risk_vector(A_enriched)
   A_enriched.risk := R
   state := Scored
```

```
// Step 4: Evaluate (policy engine / OPA)
decision := evaluate_policies(A_enriched) // allow, deny, sanitize,
   escalate
state := Evaluated

// Step 5: Transition based on decision
switch decision.type:
   case "allow":
      state := Allowed
      result := execute_action(A_enriched.payload)
   case "deny":
      state := Denied
      result := make_error("action_denied", decision.reason)
   case "sanitize":
      A_sanitized := apply_sanitization(A_enriched, decision.
         transforms)
      state := Sanitized
      result := execute_action(A_sanitized.payload)
   case "escalate":
      state := Escalated
      result := enqueue_for_human_review(A_enriched, decision)
   default:
      state := Denied
      result := make_error("invalid_decision_type", null)

// Step 6: Audit
write_audit_event(A_enriched, decision, state, result)

return result
```

E.2 FSM Transition Function

```
function fsm_transition(state, decision_type):
   if state == Proposed:
      return Scored // after scoring step

   if state == Scored:
      return Evaluated // after policy evaluation

   if state == Evaluated:
      if decision_type == "allow":
         return Allowed
      if decision_type == "deny":
         return Denied
      if decision_type == "sanitize":
         return Sanitized
      if decision_type == "escalate":
         return Escalated

   // Terminal or invalid combinations
   return state
```

Appendix F: Extended Rego Policy Snippets

This appendix contains extended Rego examples that implement common patterns: multi-tenant scoping, escalation rules, and rate-limiting for high-risk tools.

F.1 Multi-Tenant Scoping Rule

```
package ai.policy.tenant

tenant_allowed {
  input.context.tenant == "corp-eu-finance"
  input.context.environment == "prod"
}

deny[msg] {
  not tenant_allowed
  msg := sprintf("Tenant/environment combination not allowed: %v/%v",
              [input.context.tenant, input.context.environment])
}
```

F.2 Risk-Based Escalation for Financial Tools

```
package ai.policy.finance

import future.keywords.in

high_risk_tool := tool {
  tool := input.payload.tool
  tool in {"execute_trade", "modify_risk_limits", "withdraw_funds"}
}

escalate[msg] {
  high_risk_tool
  risk := input.risk.combined
```

```
  risk >= 0.20
  risk < 0.40
  msg := sprintf("Financial tool '%v' requires human approval (risk=%.2f
     )",
              [input.payload.tool, risk])
}

deny[msg] {
  high_risk_tool
  input.risk.combined >= 0.40
  msg := sprintf("Financial tool '%v' blocked (risk too high: %.2f)",
              [input.payload.tool, input.risk.combined])
}
```

F.3 Rate Limiting and Burst Control

```
package ai.policy.ratelimit

import data.rate_counters

// Example: at most 10 executions per minute, per principal, per tool.
deny[msg] {
  tool := input.payload.tool
  principal := input.principal

  count := rate_counters[principal][tool].last_60s
  count >= 10

  msg := sprintf("Rate limit exceeded for %v on tool %v (count=%v)",
```

```
            [principal, tool, count])
}
```

Appendix G: Additional Diagrams and Classifier Configuration

This appendix provides an additional high-level diagram and a summary of safety classifier configuration.

G.1 High-Level C4-Style Context Diagram

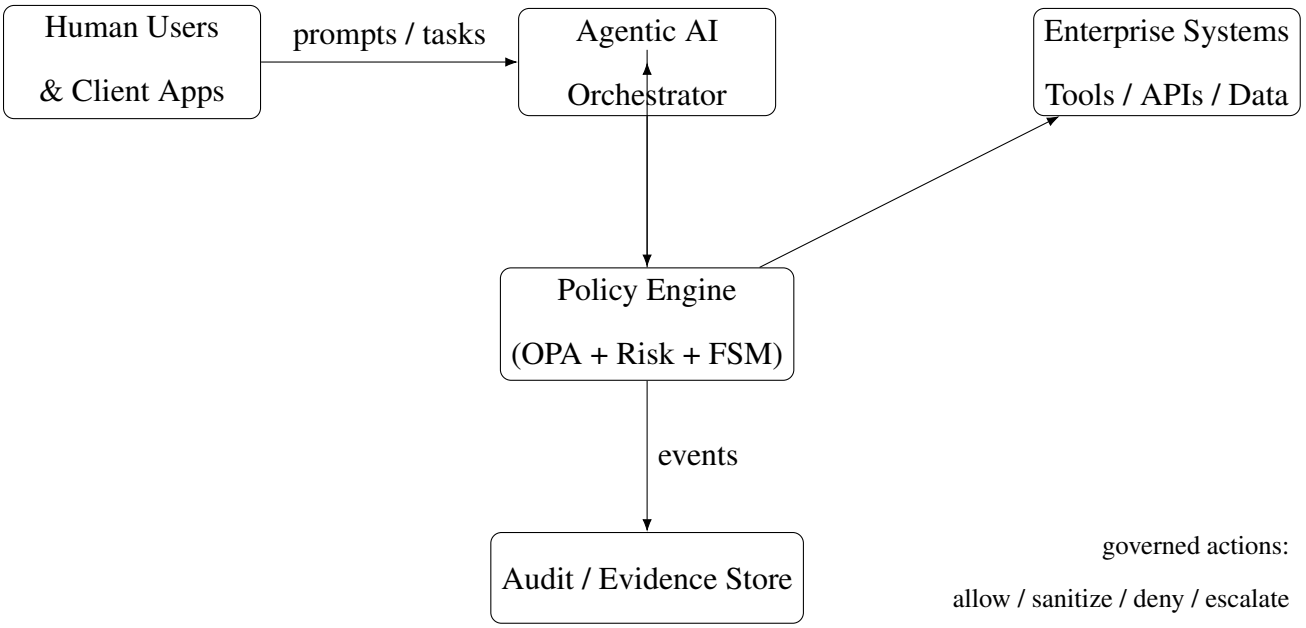


Figure 6: Context-level view of the policy engine within the enterprise architecture.

G.2 Example Classifier Configuration Table

Table 4 provides an example configuration for the six core safety classifiers used in the prototype implementation.

Table 4: Example safety classifier configuration.

Classifier	Domain	Latency (ms)	Threshold	Critical?
PII Detector	Privacy	8	0.70	Yes
Toxicity Filter	Safety	12	0.60	Medium
Jailbreak Detector	Security	6	0.55	Yes
Hallucination Model	Reliability	18	0.50	Medium
Domain Risk (Fin.)	Finance	15	0.45	High
Anomaly Detector	Behavior	10	0.65	Medium

In deployments with stricter latency budgets, the set of active classifiers can be tuned per tool or per environment, while keeping PII and jailbreak detectors enabled as critical components of the defense-in-depth strategy.

AI Tools and Responsibility Statement

The author used generative artificial intelligence tools (including large language models) to assist with drafting, editing, and formatting portions of this manuscript. These tools were employed as writing aids and for code or configuration snippet generation, but all substantive ideas, system designs, legal and policy interpretations, and conclusions are the author's own. The author independently reviewed, verified, and, where necessary, corrected any AI-generated content before inclusion. No confidential, proprietary, or personally identifiable information was intentionally submitted to AI tools during this process. The author assumes full responsibility for the accuracy, integrity, and originality of the work presented here, and any errors or omissions remain solely the author's.

References

[1] Ebers, M. (2025). *Truly risk-based regulation of artificial intelligence: How to implement the EU's AI Act*. European Journal of Risk Regulation, 16(2), 684–703.

[2] R. Sousa Silva, "Overview of the EU AI Act," *Computer Law & Security Review*, 2025. Sousa e Silva, N. (2025). *The Artificial Intelligence Act: Critical Overview*. *J. Intell. Prop. Info. Tech. & Elec. Com. L.*, 16, 2.

[3] Benraouane, S. A. (2024). *AI Management System Certification According to the ISO/IEC 42001 Standard: How to Audit, Certify, and Build Responsible AI Systems*. Productivity Press.

[4] AI, N. (2023). *Artificial Intelligence Risk Management Framework (AI RMF 1.0)*. Retrieved from https://nvlpubs.nist.gov/nistpubs/ai/NIST.AI.100-1.

[5] AI, N. (2024). *Artificial Intelligence Risk Management Framework: Generative Artificial Intelligence Profile*. NIST Trustworthy and Responsible AI, Gaithersburg, MD, USA.

[6] Habbal, A., Ali, M. K., & Abuzaraida, M. A. (2024). *Artificial Intelligence Trust, Risk and Security Management (AI TRiSM): Frameworks, Applications, Challenges and Future Research Directions*. Expert Systems with Applications, 240, 122442.

[7] Peng, B., Chen, K., Niu, Q., Bi, Z., Liu, M., Feng, P., … & Yin, C. H. (2024). *Jailbreaking and Mitigation of Vulnerabilities in Large Language Models*. arXiv preprint arXiv:2410.15236.

[8] Dong, Y., Mu, R., Zhang, Y., Sun, S., Zhang, T., Wu, C., … & Huang, X. (2025). *Safeguarding Large Language Models: A Survey*. Artificial Intelligence Review, 58(12), 382.

[9] Bommasani, R. (2021). *On the Opportunities and Risks of Foundation Models*. arXiv preprint arXiv:2108.07258.

[10] Achiam, J., Adler, S., Agarwal, S., Ahmad, L., Akkaya, I., Aleman, F. L., … & McGrew, B. (2023). *GPT-4 Technical Report*. arXiv preprint arXiv:2303.08774.

[11] Yao, S., Zhao, J., Yu, D., Du, N., Shafran, I., Narasimhan, K. R., & Cao, Y. (2022, October). *ReAct: Synergizing Reasoning and Acting in Language Models*. In *The Eleventh International Conference on Learning Representations (ICLR)*.

[12] Schick, T., Dwivedi-Yu, J., Dessì, R., Raileanu, R., Lomeli, M., Hambro, E., … & Scialom, T. (2023). *Toolformer: Language Models Can Teach Themselves to Use Tools*. Advances in Neural Information Processing Systems, 36, 68539–68551.

[13] Liu, X., Yu, H., Zhang, H., Xu, Y., Lei, X., Lai, H., … & Tang, J. (2023). *AgentBench: Evaluating LLMs as Agents*. arXiv preprint arXiv:2308.03688.

[14] Karpas, E., Abend, O., Belinkov, Y., Lenz, B., Lieber, O., Ratner, N., … & Tenenholtz, M. (2022). *MRKL Systems: A Modular, Neuro-Symbolic Architecture that Combines Large Language Models, External Knowledge Sources and Discrete Reasoning*. arXiv preprint arXiv:2205.00445.

[15] Floridi, L. (2023). *The Ethics of Artificial Intelligence: Principles, Challenges, and Opportunities*.

[16] Akbarighatar, P. (2025). *Operationalizing Responsible AI Principles Through Responsible AI Capabilities*. AI and Ethics, 5(2), 1787–1801.

[17] Amodei, D., Olah, C., Steinhardt, J., Christiano, P., Schulman, J., & Mané, D. (2016). *Concrete Problems in AI Safety*. arXiv preprint arXiv:1606.0656

[18] Xia, B., Lu, Q., Zhu, L., & Xing, Z. (2024, July). *An AI System Evaluation Framework for Advancing AI Safety: Terminology, Taxonomy, Lifecycle Mapping*. In *Proceedings of the 1st ACM International Conference on AI-Powered Software* (pp. 74–78).

[19] Bai, Y., Kadavath, S., Kundu, S., Askell, A., Kernion, J., Jones, A., … & Kaplan, J. (2022). *Constitutional AI: Harmlessness from AI Feedback*. arXiv preprint arXiv:2212.08073.

[20] Salhab, W., Ameyed, D., Jaafar, F., & Mcheick, H. (2024). *A Systematic Literature Review on AI Safety: Identifying Trends, Challenges and Future Directions*. IEEE Access.

[21] Jayaraman, K. D. (2025). *AI Governance in Cloud-Native Systems: Embedding Policy-Driven Controls in Enterprise Platforms*.

[22] Agent, O. P. (2023). *Policy-Based Security and Compliance for Cloud Native Environments*. OPA Documentation (Project documentation), OPA website.

[23] Adeyinka, A. (2023). *Automated Compliance Management in Hybrid Cloud Architectures: A Policy-as-Code Approach*.

[24] Liu, Y., Yao, Y., Ton, J. F., Zhang, X., Guo, R., Cheng, H., … & Li, H. (2023). *Trustworthy LLMs: A Survey and Guideline for Evaluating Large Language Models' Alignment*. arXiv preprint arXiv:2308.05374.

[25] Topsakal, O., & Akinci, T. C. (2023, July). *Creating Large Language Model Applications Utilizing LangChain: A Primer on Developing LLM Apps Fast*. In *International Conference on Applied Engineering and Natural Sciences* (Vol. 1, No. 1, pp. 1050–1056).

[26] Pelluru, K. (2025). *LangChain & LangGraph in Production: Architectures for Multi-Agent LLM Systems*. Journal of Data and Digital Innovation (JDDI), 2(3), 1–9.

[27] Baier, C., & Katoen, J. P. (2008). *Principles of Model Checking*. MIT Press.

[28] Perez, F., & Ribeiro, I. (2022). *Ignore Previous Prompt: Attack Techniques for Language Models*. arXiv preprint arXiv:2211.09527.

[29] Gulyamov, S., Gulyamov, S., Rodionov, A., Khursanov, R., Mekhmonov, K., Babaev, D., & Rakhimjonov, A. (2025). *Prompt Injection Attacks in Large Language Models and AI*

Agent Systems: A Comprehensive Review of Vulnerabilities, Attack Vectors, and Mitigation Strategies.

[30] Rahman, M. A., Shahriar, H., Francia, G., Wu, F., Cuzzocrea, A., Rahman, M., … & Ahamed, S. I. (2025, July). *Fine-Tuned Large Language Models (LLMs): Improved Prompt Injection Attacks Detection*. In *2025 IEEE 49th Annual Computers, Software, and Applications Conference (COMPSAC)* (pp. 1033–1039). IEEE.